Train-Test Splits

2026-04-17

Introduction

A train-test split reserves a random subset of the data for honest evaluation of model generalisation, separating it cleanly from the data used for fitting and tuning. It is the simplest possible evaluation protocol and the foundation of every supervised-learning workflow. The trade-off is straightforward: a larger test set gives more stable error estimates but leaves less data for fitting; for small datasets, cross-validation extracts more value from the available data than a single fixed split.

Prerequisites

A working understanding of supervised learning, the train-validate-test paradigm, and the difference between hyperparameter tuning and final model evaluation.

Theory

With n observations, reserve n_{test} for test. Typical ratios are 80/20, 70/30, or 90/10 depending on dataset size. The test-set error is a point estimate of generalisation with variance $O(1/n_{\text{test}})$ — for stable error estimates the test set should be at least a few hundred observations.

Stratified splits sample within each class so class proportions match between train and test; this is essential for imbalanced classification, where random sampling can leave the minority class severely under- or over-represented in either fold.

Grouped splits keep all observations from the same subject, site, or sample together to prevent leakage; ignoring grouping when it exists silently inflates performance estimates.

Assumptions

Observation pairs (x_i, y_i) are independent and identically distributed (or the dependence structure is respected by the chosen split scheme); deployment distribution matches the training distribution.

R Implementation

```
library(rsample); library(dplyr)

set.seed(2026)
n <- 1000
df <- tibble(x = rnorm(n),
             y = factor(sample(c("pos", "neg"), n,
                               replace = TRUE, prob = c(0.1, 0.9))))

# Random 80/20 split
split1 <- initial_split(df, prop = 0.8)

# Stratified split preserving y proportions
split2 <- initial_split(df, prop = 0.8, strata = y)
```

```r
tbl_noStrata <- table(training(split1)$y) / nrow(training(split1))
tbl_strata   <- table(training(split2)$y) / nrow(training(split2))

round(rbind(noStrata = tbl_noStrata, strata = tbl_strata), 3)
```

Output & Results

The proportion table shows that stratified splitting preserves the 10 % minority-class proportion in both train and test; random splitting can drift by 2–3 percentage points, especially with smaller test sets. `training(split)` and `testing(split)` extract the two folds.

Interpretation

A reporting sentence: "An 80/20 stratified split preserved the 10 % minority-class proportion in both training and test folds (training: 10.1 %, test: 9.9 %); we tuned hyperparameters on the training set via 5-fold cross-validation and reserved the test set for a single final evaluation, never touching it during model development." Always state the split ratio, stratification, and how the test set was used.

Practical Tips

- For time series, split chronologically (`rsample::initial_time_split()`); random splitting leaks future information into training and produces optimistic estimates.
- For grouped data (multiple observations per subject), use `rsample::group_initial_split()` to prevent leakage; ignoring grouping when it exists silently inflates performance estimates.
- Test sets should be large enough for stable error estimates; below 200 observations the test error has high variance and a single split may not generalise.
- The test set is touched *once*: after model development is complete. Repeated use during development implicitly tunes against it and produces optimism that no single number can capture.
- For final deployment, refit the best model on the training and test sets combined; the test set served its purpose at evaluation time and there is no reason to discard its information.
- For small datasets, prefer cross-validation over a single train-test split; CV uses the data more efficiently and gives a less variable error estimate.

R Packages Used

`rsample::initial_split()` and `initial_time_split()` for splits; `group_initial_split()` and `group_vfold_cv()` for grouped data; `caret::createDataPartition()` for an older alternative; `mlr3::partition()` for mlr3-style splits; base R `sample()` for ad-hoc splitting.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting

- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis